STM32 Firmware Technical Documentation

Reconfigurable Digital Integrated Circuit Test Fixture

---

1. Overview

This document provides a technical overview of the STM32 firmware developed for the Reconfigurable Digital Integrated Circuit Test Fixture. It describes the architecture, core modules, communication protocol, and key implementation details required for system operation.

The firmware is responsible for:

- Receiving test commands via UART
- Configuring hardware routing through analog switch arrays
- Driving and stimulating IC inputs
- Measuring outputs via ADC
- Performing fault detection and validation
- Executing automated test sequences

The firmware operates as the central controller between the GUI application and the physical test hardware.

---

2. System Architecture

2.1 High-Level Operation

The firmware follows a command-driven architecture:

Figure 1. High-Level Firmware Flow

Start → UART Receive → Parse Command → Update State
→ (TEST/BIST/CRC trigger)
→ Execute Operation
→ Send Results via UART
→ Repeat

Execution is event-based using flags set during UART command parsing.

2.2 Core Firmware Files

File	Description
main.c	Entry point, UART handling, command parsing, system control
test_utils.c	Core test execution logic and routing control
fault_handling.c	Preflight validation and fault detection
nau7802.c	ADC interface and measurement
mux_utils.c	Output selection mux control
adg2128_router.c	Switch matrix definitions (used by test_utils)
bist.c	Built-in self test functionality
firmware_crc.c	Firmware integrity verification

---

3. Command Interface (UART Protocol)

The firmware communicates using a simple ASCII command protocol.

3.1 Command Format

COMMAND:parameters\n

Example:

PRM:20,10,5
INS:1,2,3
VIP:0b1010,0b0101
TEST

3.2 Supported Commands

Command	Description
PING	Connectivity check
START	Marks beginning of test script
PRM	Define VCC pin, GND pin, voltage
VIN	Define logic thresholds
INS	Define input pins
OUT	Define output pins
VIP	Define input patterns
TEST	Execute test
BIST	Run hardware self-test
FWCRC	Compute firmware checksum

3.3 Command Processing Flow

Inside main.c, commands are parsed and mapped to flags:

- g_run_test
- g_run_bist
- g_start_script
- g_crc_start

Execution occurs in the main loop after parsing .

---

4. Test Execution Pipeline

4.1 Test Flow

Figure 2. Test Execution Flow

TEST Command →
(if START)
→ ADC Init
→ ADC Calibration
→ Fault Preflight
→ Apply Inputs
→ Generate Clock Events
→ Read Outputs
→ Report Results

4.2 Session Optimization

The firmware uses **session caching**:

- Expensive operations (ADC calibration, fault checks) run once per script
- VIP changes do NOT trigger recalibration

This significantly improves performance.

---

5. Hardware Control

5.1 Switch Matrix (ADG2128)

The firmware controls two ADG2128 crosspoint switches to route signals.

Key features:

- Each DUT pin maps to a switch X-line
- Voltage sources map to Y-lines
- Routing performed via I2C

Example mapping logic :

- Pins 1–10 → Chip 1
- Pins 11–20 → Chip 2

5.2 Routing Sources

Source	Description
1.8V – 5V	Voltage rails
STM32 GPIO	Digital driving
GND	Ground reference

Routing is dynamically configured per test step.

5.3 Clock Generation

Clock signals are generated via an STM32 GPIO:

pulse_stm32_source_gpio(low, high, final_low);

Sequence:

1. Drive LOW
2. Drive HIGH
3. Return LOW

Timing is configurable and critical for flip-flop testing.

5.4 Clock Voltage Matching

A mux ensures the clock voltage matches VCC:

- Controlled via GPIO (PB0–PB2)
- Prevents logic-level mismatch

---

6. Input Pattern Execution

6.1 VIP Input Types

Type	Description
Voltage	Fixed level (e.g., 3.3V)
Binary	Serial pattern (0b1010)
CLK	Clock-driven input

6.2 Serial Pattern Handling

Binary inputs support:

- 0 → LOW
- 1 → HIGH
- R → Rising edge
- F → Falling edge

Example:

0b10RF

This allows precise control of sequential logic.

6.3 Step Execution

Each test step:

1. Apply inputs
2. Execute clock pulse (if needed)
3. Wait for settle
4. Read outputs

---

7. Output Measurement (ADC)

7.1 ADC Overview

The NAU7802 ADC measures DUT output voltages.

7.2 Calibration Process

Performed once per session:

1. Route GND to calibration pin
2. Take multiple samples
3. Compute offset
4. Store correction value

Example output:

ADC calibration raw=54 mV, offset=54 mV

7.3 Measurement Process

For each output:

1. Select via mux
2. Allow settling
3. Take multiple samples
4. Use median value
5. Apply calibration offset

7.4 Logic Determination

Condition	**Result**
≤ LOW threshold	0
≥ HIGH threshold	1
Between thresholds	Invalid

Condition	Result
Mid-range (~1.6V)	High-Z

---

8. Fault Handling & Preflight

8.1 Purpose

Preflight ensures:

- IC is inserted correctly
- No shorts or floating pins
- Hardware is functioning

8.2 Preflight Steps

1. Clear all routing
2. Apply power (VCC/GND)
3. Drive all inputs LOW
4. Measure outputs
5. Drive all inputs HIGH
6. Measure outputs again
7. Detect anomalies

8.3 Controlled vs Passive Modes

- Passive scan used for certain IC types
- Avoids driving outputs incorrectly

8.4 Failure Handling

If fault detected:

- Test is aborted
- Error reported via UART

---

9. Built-In Self Test (BIST)

The BIST verifies hardware integrity:

- Switch matrix functionality
- Voltage routing
- ADC behavior

Triggered by:

BIST

After BIST:

- Test session is invalidated

---

10. Firmware Checksum (CRC)

10.1 Purpose

Ensures firmware integrity and correct deployment.

10.2 Implementation

Triggered via:

FWCRC

Returns:

FW CRC32=0xXXXXXXXX

Computed over firmware memory using CRC32.

---

11. Performance Considerations

Key optimizations:

- Session caching (avoids repeated calibration)
- Median filtering for ADC
- Selective routing updates (no redundant switching)
- Asynchronous UART handling

---

12. Limitations & Future Work

Current Limitations

- Limited voltage levels
- Fixed ADC sampling strategy
- No adaptive timing for different ICs

Potential Improvements

- Dynamic timing adjustment
- Expanded voltage support
- Faster ADC sampling
- Enhanced fault diagnostics
- Parallel test execution

---

13. Conclusion

The STM32 firmware provides a robust and flexible platform for automated IC testing. Its modular architecture enables:

- Reliable hardware control
- Accurate measurement
- Efficient test execution
- Strong integration with GUI software

The system is designed to closely resemble real production test environments, improving usability for technicians and ensuring high-quality validation of digital ICs.